Hardware Trust Anchor Authentication for Updatable IoT Devices

Dominik Lorych
dominik.lorych@sit.fraunhofer.de
Fraunhofer SIT | ATHENE
Darmstadt, Hesse, Germany

Christian Plappert
christian.plappert@sit.fraunhofer.de
Fraunhofer SIT | ATHENE
Darmstadt, Hesse, Germany

ABSTRACT

Secure firmware update mechanisms and Hardware Trust Anchors (HTAs) are crucial in securing future IoT networks. Among others, HTAs can be used to shield security-sensitive data like cryptographic keys from unauthorized access, using hardware isolation. Authentication mechanisms for key usage, however, are difficult to implement since corresponding credentials need to be stored outside the HTA. This makes them vulnerable against host hijacking attacks, which in the end also undermines the security gains of the HTA deployment.

This paper introduces an update-resilient and secure HTA authentication mechanism that secures the HTA authentication credentials on the host. Our concept is based on an integration of the Device Identifier Composition Engine (DICE), a Trusted Computing standard for resource-constrained off-the-shelf devices, with signed update manifest documents. This secures HTA authentication credentials, but also provides value for DICE-based devices without an HTA. We evaluate the feasibility of our solution based on a proof-of-concept implementation.

CCS CONCEPTS

• **Security and privacy** → **Trusted computing**; *Embedded systems security*; Authentication.

KEYWORDS

Hardware Trust Anchor (HTA), authentication, Device Identifier Composition Engine (DICE), firmware updates, SUIT manifests

ACM Reference Format:
Dominik Lorych and Christian Plappert. 2024. Hardware Trust Anchor Authentication for Updatable IoT Devices. In *The 19th International Conference on Availability, Reliability and Security (ARES 2024), July 30–August 02, 2024, Vienna, Austria.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3664476.3664479

1 INTRODUCTION

The current development to increased connectivity across all kinds of devices causes challenges for the security of devices, especially for resource-constrained devices. This development also includes safety-critical areas (e.g., automotive and rail) or critical infrastructure (e.g., smart factories, smart grid), where security and safety is critical. Here Hardware Trust Anchors (HTAs) can be used, which increase security by hardware-isolating critical parts like cryptographic algorithms and their keys and which often are tamperproof and side-channel resistant. In some of these critical areas, low-level HTAs have been state of the art for a while (e.g., the Secure Hardware Extension (SHE) and the Evita Hardware Security Modules (HSMs) in the automotive sector), but the adoption of higher-level HTAs is increasing. Low-level HTAs often only support symmetric cryptography due to size and speed constraints, while the higher-level HTAs usually also support asymmetric cryptography.

However, authentication of these HTAs is often difficult, especially on updatable systems. Often, either no authentication or authentication using credentials saved in non-volatile memory of the host is used. This means that while an attacker cannot access keys secured in the HTA directly, they can issue commands for cryptographic operations using these keys, effectively giving them temporary access to the keys as long as they control the host Microcontroller Unit (MCU).

The Trusted Platform Module (TPM) is an HTA which offers an improved authentication mechanism. The TPM is openly specified by the Trusted Computing Group (TCG) and used in consumer electronics, e.g., personal computers and servers. Besides regular PIN/password-based authentication mechanisms, it also offers session-based authentication mechanisms that establish secure remote sessions up into the TPM. These sessions can be used to securely authenticate towards the TPM from a remote host over unsecure channels. Moreover, on consumer electronics the TPM is broadly used to authenticate the boot process, e.g., BitLocker [22]/LUKS for disk encryption. However, on many resource-constrained devices, the TPM and its corresponding boot authentication mechanisms are unsuitable due to its size, hardware requirements and cost. Moreover, if session-based authentication mechanisms are used on the same host without boot process authentication, the storage of the authentication credentials offer the same cyberattack surface as the other HTAs.

This paper aims to introduce improvements for HTA-enabled devices which cannot use a TPM. Currently, these usually do not support authentication. To keep hardware requirements for the MCU small, we are basing our concept on Device Identifier Composition Engine (DICE), as we do not expect the MCU to have much hardware security features if critical functionality is exported to an HTA. DICE is an alternative to the TPM for resource-constrained devices, also specified by the TCG. It essentially is a secure bootloader establishing a static Root of Trust, enabling the creation of a Chain of Trust architecture. Furthermore, its hardware requirements are minimal, meaning that it can be implemented on most off-the-shelf MCUs.

We use DICE-based Remote Attestation to implement authentication for the HTA. Remote Attestation enables the HTA to verify firmware running on the MCU by verifying the Chain of Trust

This work is licensed under a Creative Commons Attribution International 4.0 License.

ARES 2024, July 30–August 02, 2024, Vienna, Austria
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1718-5/24/07
https://doi.org/10.1145/3664476.3664479

created by DICE. However, we propose a new Remote Attestation method for DICE as the existing ones are either too slow for our use case or incompatible with updates.

To implement updates, we use manifests, which are metadata documents, in this case for update processing and installation. We base our implementation on SUIT Manifests, a manifest document format intended for enabling secure updates for resource-constrained devices, but other types of manifests also could be used. In our case, we additionally integrate these manifests in the Chain of Trust and use them as endorsements for the HTA, i.e.; they are used for proving to the HTA that the current firmware is authorized for the device.

Integrating a standardized update manifest like SUIT Manifests into a DICE architecture makes the implementation of updates easier as they define a standardized and generalized way of installing updates. Both the manifests themselves and the manifest processor can be used for DICE-enabled devices and non-DICE devices in the same way. Additionally, manifests can be used to ensure that only authorized application firmware can run on the device.

Contribution. In short, our contribution consists of the following:

(1) Introduction of a concept to integrate DICE with update manifest documents, including their integration into the DICE certificate chain used for asymmetric attestation
(2) Adoption of DICE remote attestation for HTA authentication, which breaks some assumptions made in usual remote attestation systems, also including usage of manifests as endorsements for the HTA
(3) Introduction of hybrid attestation, which allows updatable systems to use DICE attestation if asymmetric attestation is too slow for the use case

This work starts with more background information in Section 2. Then we explain our concept in Section 3, followed by implementation details in Section 4. The evaluation in Section 5 evaluates performance characteristics of our implementation, but also evaluates the security and feasibility of the solution. Lastly, a conclusion in Section 6 ends this paper.

2 BACKGROUND AND RELATED WORK

In this section we introduce DICE, related Trusted Computing concepts, manifest documents and update terminology. Afterward, we explain our attacker model and related work.

2.1 DICE & DICE Layering Architecture

DICE [9] is a secure bootloader for resource-limited devices like MCUs. Its task is the creation of a static Root of Trust as the foundation of a Chain of Trust. Here, the Chain of Trust is represented by a chain of keys representing the device identity and firmware integrity, meaning changes to either device or firmware will result in a different set of keys. This means that changes to the system can be detected during the boot, establishing trust in the system. As the Root of Trust, DICE is required to be immutable.

A DICE layering architecture [12] consists of multiple layers that are executed sequentially during the boot. The task of DICE is to compute the first Compound Device Identifier (CDI), where the CDIs are the cryptographic keys representing the Chain of Trust, using a one-way function like Hash-based Message Authentication Code (HMAC) [2, 5]: $CDI_{L_0} = HMAC(UDS, TCI_{L_0})$. Here the Unique Device Secret (UDS) is a secret exclusively known by DICE to ensure that only DICE can compute this key. The Trusted Computing Base (TCB) Component Identifier (TCB) identifies and represents firmware, in this case the firmware executed after DICE. It is basically a hash of the firmware, optionally complemented with some additional identifying information.

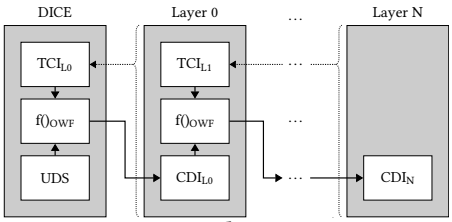

Figure 1: DICE Architecture Overview

During the boot of other components, the CDI gets extended with the TCIs of the other firmware layers until the CDI represents all firmware components: $CDI_{L_{i+1}} = HMAC(CDI_{L_i}, TCI_{L_{i+1}})$. This construction ensures that neither a different device with the same firmware nor the same device with different firmware can compute those CDIs, enabling it to be used as the foundation of trust. An overview of the architecture is shown in Fig. 1.

The disadvantage of those CDIs is that updates cannot be verified externally as there is no way to detect whether a changed CDI is due to a malicious or intended update. This can be changed by instantiating an Embedded Certificate Authority (ECA) in the firmware executed after DICE (Layer 0 (L0)), based on two asymmetric keys [10]: The DeviceID key is the private key of the ECA that represents the identity of the ECA, derived from the CDI_{L0}. It does not change as long as L0 does not need to be updated. The Alias key is also created in L0, but represents the application, so it changes with every update to it. The ECA signs a certificate of the Alias key using its DeviceID key, which among others contains the TCI of the application. This certificate can be used to verify which application is currently running on the device, given that the DeviceID is trusted.

2.2 Trusted Computing Concepts

In the context of DICE, Remote Attestation and Sealing are interesting concepts of Trusted Computing.

2.2.1 Remote Attestation. Based on the Chain of Trust created by DICE, Trusted Computing techniques can be implemented on MCUs. One of them is Remote Attestation, which enables external parties to verify the trustworthiness of a device. Remote Attestation is a process between a party that wants to prove its trustworthiness, the attester, and a party verifying trustworthiness, the verifier. Additional roles may also be involved [14], for example the relying party, which relies on the attester and trusts the verifier to correctly evaluate trustworthiness.

DICE can be used to enable an attester to prove its trustworthiness based on the keys created during the boot. Here a proof of possession of a specific key is used, usually either a symmetric key derived from the last CDI or the Alias key. Attestation using

asymmetric keys [10] is slow, but can be integrated more easily on updatable systems. Changes to the system can be verified using the Alias certificate, while the proof of possession is done using a signature. Attestation based on symmetric keys [13] is much faster, but changes after an update can only be verified when they have been done in a secure environment as there is no way to externally verify changes of the CDIs. Here the proof of possession is done using an HMAC.

Remote Attestation should be complemented with endorsements, which are information provided by the manufacturers and suppliers that describe the attester, intended to help the verifier to decide whether the attester can be trusted. The TCG has created an endorsement architecture [15] that provides guidance for this.

2.2.2 Sealing. Sealing describes the encryption and authentication of data using a key that is bound to the device state. In the case of DICE, a CDI-derived key is used. This can be used to securely save sensitive or secret data in the flash memory, so that manipulated devices cannot access the data. However, the simplest form of this concept again does not work well with updates as all data will be lost after updates. Data can be migrated before updates to make it accessible afterward, but this is often tedious. Some migration techniques are described within the predecessor of DICE [7].

Another usage for sealing is the caching of asymmetric keys and certificates. Usually, all asymmetric keys generated by L0 need to be generated on every boot, which takes a long time on low-level MCUs. By sealing those keys and certificates using a symmetric key that is derived from the same inputs as the asymmetric key, the generation of those keys and certificates can be avoided at the next boot given the system did not change. As the sealing key is derived from the same inputs as the keys and certificates contained in the sealed data, the sealed data can only be decrypted if the system is in a state where the keys and certificates also could be generated normally. This accelerates the boot of a DICE system considerably as generation of asymmetric keys and certificates usually is by far the biggest factor on the boot times of a DICE system. The approach can be used, for example, in the automotive domain, where each MCU usually is required to boot in less than half a second.

2.3 Manifests

The term manifest describes a metadata document, in the context of our paper used to refer to metadata documents for firmware. One instance of this are SUIT Manifests, currently in the process of standardization in the Internet Engineering Task Force (IETF) workgroup Software Updates for Internet of Things (SUIT). Their information model is published as an informational RFC [23], based on an update architecture for Internet of Things (IoT) environments [25]. A SUIT Manifest serialization format [24] based on the Concise Binary Object Representation (CBOR) [6] and CBOR Object Signing and Encryption (CBOR) [27] is currently in the late stages of the draft phase, intended to be published as a proposed standard. SUIT Manifests contain information about how to process the update and additional information that can be used to check whether the update applies to the device that received the manifest. This is wrapped by an authentication wrapper containing a signature of the creator of the manifest, used to verify the integrity of the manifest and whether a trusted party created the update. Manifests are designed

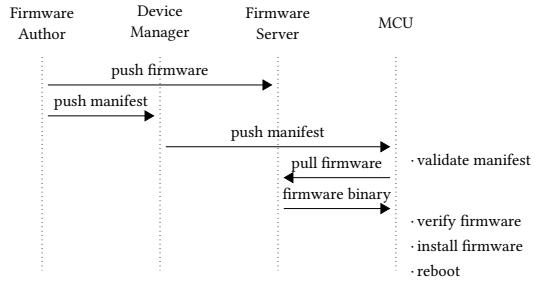

Figure 2: Simplified Update Procedure

to be simple to parse and process on constrained systems, while still being expressive and extensible enough to support more advanced use cases. A simpliefied update procedure for a manifest is shown in Fig. 2.

Trusted Invocation (i.e., Secure Boot and similar mechanisms of that only execute trusted firmware) is also supported as it tends to use a subset of the operations and data needed for software updates, so this can be included without much overhead. Additional features are dependencies to other manifests and severable elements. The latter are parts that can be removed from the manifest without impacting the signature of the manifest, while still being authenticated when included in the manifest. An example here would be a human-readable text field as a description that can be removed before sending the manifest to the MCU to reduce manifest size. To enable recipients of severed manifests to retrieve the full manifest, a reference Uniform Resource Identifier (URI) to the full manifest may be included in a manifest. Each manifest must contain a sequence number, which is a monotonically increasing number used as anti-rollback. Each recipient of a manifest must reject the manifest if the sequence number is lower than the current one.

The concepts of this paper also could be implemented with other types of update metadata documents as long as they support the required features. We will use 'manifest' for these, while SUIT Manifests are used to refer to the specific instance.

2.4 A/B Update Scheme

A/B updates describe systems that reserve two slots for the image of the updatable component in question, where those slots are named slot A and slot B. The active slot contains the current version of the component, while the inactive one contains a previous version. Assuming slot A is the active slot when the system receives an update, then the new version gets written into slot B. Upon the next boot, the system detects the new version and boots slot B. This approach is resilient as failures during the update process (e.g., power failures) can easily be recovered by booting the older component and repeating the update. Moreover, the system can easily downgrade to the older version if the update does not work due to bugs.

Embedded systems usually use A/B update schemes as update images often are too big to keep them entirely in the RAM, so a caching area is needed to write the update image to. Here the integrity of the image can be checked, e.g., by hashing the image, before installing the image into the slot of component. When using A/B images, the slot containing the older version can be used as

caching area, so the flash size requirements stay the same. If there are multiple updatable components, manifests also could enable DICE to use shared slots for multiple components (e.g., three slots for two updatable components, with manifests identifying which slot currently holds which component). Firmware for MCUs is often not position-independent, i.e.; compiled to be executed at a specific address, which means that firmware requires separate binaries for separate slots. This also means that manifests for A/B update schemes require two separate download URIs, one for each binary.

2.5 Attacker Model

DICE is intended for low-cost, resource-constrained devices. Capabilities to defend against complex attacks are limited due to the limited hardware. Weaknesses include runtime attacks (but countermeasures exist, see [17]) and advanced physical attacks. However, DICE-enabled devices are usually low-cost devices performing tasks with small criticality, so it might not even be worth it for an attacker to execute more complex attacks.

We consider three types of attackers:

- Remote attackers according to Dolev-Yao
- Hijacking attackers that can execute malicious code on the MCU, using e.g. runtime attacks based on Return Oriented Programming (ROP)
- Physical attackers using e.g. fault injection or side-channel attacks

Of those, we are mainly interested in defending against hijacking attackers. Our concept can also defend against remote attackers, but our contribution provides no real advantage here against the state of the art. Instead, we see the value of our contribution in the defense against hijacking attackers. The authentication of the MCU stops these attackers from using keys and other secrets that are saved in the HTA. Runtime attacks are also covered as long as one of the mitigations presented in the evaluation gets used. Physical attacks are out of scope except for some simple attacks which manipulate flash memory content, as DICE cannot detect or mitigate more advanced attacks. Direct attacks against the HTA which enable the attacker to execute malicious code on the HTA are also out of scope as the HTA generally is assumed to be protected well through hardware security.

2.6 Related Work

Enhancing security by storing security-critical information like cryptographic keys in HTAs is a state-of-the-art mitigation mechanism to protect against logical software attacks and, e.g., in case of the TPM [11], against side-channel and physical attacks. HTAs are widely deployed in consumer electronics, e.g., smartphones and notebooks, but started to be also increasingly used in embedded networks and critical infrastructures like smart grids. There are various HTAs ranging from dedicated coprocessors like the TPM and on-chip security subsystems like HSMs to processor extensions like ARM TrustZone or Intel SGX. Plappert et al. [26] compare and evaluate HTAs regarding their feasibility to be used in automotive use cases. In their comparison they analyze (1) HTAs that do not provide any usage authorization, e.g., SHE [8], (2) HTAs that provide simple password-based authentication like the Evita HSMs [30], and (3) HTAs that also provide session-based authentication mechanisms, e.g., the TPM. However, while key material is securely stored in the HTAs, the HTA access is vulnerable against host compromise.

The TCG is currently working on a DICE Protection Environment (DICE) [16], which was recently in public review. Its basic idea is to generate secrets of DICE (CDIs and derived keys) in the HTA where they are protected against runtime attacks. The DPE is primarily intended for Sealing and Remote Attestation, while Authentication of the host MCU is not intended.

Regarding updates for IoT devices, Zandberg et al. have conducted a study regarding their implementation using open standards [32]. For this, they compared different levels of solutions from systems without Over-the-Air (OTA) update functionality to basic OTA update functionality and more advanced update systems based on SUIT Manifests and the Lightweight Machine-to-Machine (LwM2M) protocol, an open standard of IoT device management. They find that while the cost of doing OTA updates is relatively high due to, among others, needing two slots for the firmware image, the benefits should outweigh the costs in most situations. Introducing more advanced standards like SUIT Manifests increases security and capabilities, but again increases code size. The LwM2M protocol enables the device to better withstand offline device attacks (taking the device offline to later install a newer, but known to be flawed update) and resource exhaustion attacks (draining resources by sending SUIT Manifests with an incorrect signature).

The Uptane framework is an alternative to SUIT Manifests intended for automotive environments, as demonstrated by Kuppusamy et al. [20, 21]. It also uses signed manifest documents, but adds features especially adapted to automotive subnets like offloading manifest verification to a central MCU to reduce effort for less powerful MCUs embedded in the subnet.

On the topic of DICE, Hristozov et al. have demonstrated a Time-of-check Time-of-use (TOCTOU) attack on DICE Attestation by using a runtime attack [18]. DICE in general is vulnerable to runtime attacks due to limited hardware capabilities, hence any secrets available to the application during the runtime can be accessed by runtime attackers. In their case, they demonstrate that a runtime attacker could copy the Alias key into flash memory so that the key is available to manipulated applications. In principle, the attestation used in this paper is also vulnerable to the same attacks, but there is a simple countermeasure when used as HTA authentication, which we present in the evaluation.

As an alternative, Hristozov et al. previously proposed an expansion of DICE to be able to detect runtime attacks [17]. It enables attestation code to jump back into L0, where a fresh measurement of the application gets created, signed with an asymmetric attestation key that is only available to L0. Here an Memory Protection Unit (MPU) is required to protect secrets of L0 from accesses of the application. They implemented this on a STM32L476RG, based on an ARM Cortex-M4.

Xu et al. have introduced a new security scheme called dominance [31]. They define dominance as the ability to execute arbitrary code on a device with the guarantee that it will be executed within a certain time frame. The implementation is based on DICE, but extends L0 with a network stack and an updater and includes

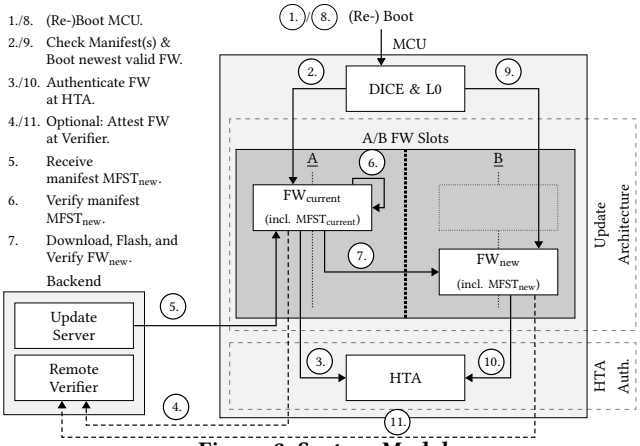

Figure 3: System Model

an Authenticated Watchdog Timer (AWDT). Later, Huber et al. presented an improvement [19] that replaces the AWDT with a normal watchdog and a component running inside an ARM TrustZone, but needed to disable deep sleep to ensure that the watchdog is always running. Both are based on forcing the device to reboot, upon which L0 checks for updates using the included network stack and updater.

3 CONCEPT

This section first outlines our high-level concept, then elaborates the three main parts of the concept in more detail and describes additional options and optimizations.

3.1 High-Level Concept

Our system model is depicted in Fig. 3 and consists of the DICE-secured MCU (attester) with an attached HTA, an update server providing update data to the MCU, and optionally an external verifier that appraises the trustworthiness of the MCU, e.g., that it runs up-to-date software. The MCU has access to a dedicated HTA, e.g., a dedicated hardware security chip like an Secure Element (SE) or a Smartcard, or an on-chip security subsystem like an HSM. Here the MCU attests to the HTA and a successful attestation would, e.g., grant the MCU to access cryptographic keys of the HTA.

Our core concept can be divided into three main parts, (1) the update architecture for MCU firmware, (2) the DICE-secured boot mechanism of the MCU, and (3) the attestation architecture between MCU and HTA. Here, the update architecture is responsible to provide secure updates to the application, based on manifests. The DICE-secured mechanism is the foundation for enabling Remote Attestation, but also ensures that only signed firmware images get access to sealed data saved by the MCU, also based on manifests. The attestation architecture ensures that the HTA can verify the MCU's trustworthiness, with manifests providing endorsements for the HTA (i.e.; enabling the HTA to decide whether a previously unknown firmware is trustworthy).

The integration of those parts is shown in Fig. 3. Initially, there is only a single firmware installed on the system ($FW_{current}$). The system first boots DICE and L0, with the latter checking the manifests of both firmware slots (2). Only slot A contains a valid manifest and firmware, so it gets booted. The application authenticates itself at the HTA (3). Optionally, the firmware can also be attested by an external verifier (4). Now the application starts its normal operation. At some point, it receives a new manifest $MFST_{new}$ (5). This manifest gets verified by the application (6). If it is valid, the binary for the new firmware FW_{new} gets downloaded, installed in the unused slot B and verified (7). If the firmware image is valid, the system will be rebooted (8). Here, both $MFST_{current}$ and $MFST_{new}$ get verified, with the result that $MFST_{new}$ is newer and valid (9). The firmware FW_{new} gets booted and authenticates itself at the HTA. Optionally, the firmware can also be verified by an external verifier.

3.2 Update Architecture

Our update architecture follows the A/B slot structure. Each slot holds one manifest corresponding to the currently installed firmware and the firmware binary. The manifests are used to verify updates before installing them and to verify firmware integrity before booting them. SUIT Manifests have the advantage that the steps of manifest creation, update delivery, update processing and installation can be shared across DICE and non-DICE devices, which unifies the architecture and makes adoption of DICE into an existing update infrastructure more likely.

During an update, the MCU first receives a manifest from the update server, which gets parsed and validated. If this is successful, the MCU will try to download the firmware image referenced in the manifest. A manifest for A/B images will contain two URIs, where the MCU downloads the correct one based on the chosen slot for installation. This is required because firmware images for IoT devices are usually not position-independent, i.e.; they can only be executed if they are installed at a specific memory location. During download, the image gets written into the slot and then hashed for verification. If the image matches the given hash of the manifest, the manifest also gets written into flash memory so that it can be verified during the boot. Then the system reboots to run the new image.

3.3 DICE-based Boot System

The boot system uses DICE and L0, with manifests being incorporated into the latter. L0 has the task of checking both slots of the application using their manifests. It parses both manifests, verifies their validity and the integrity of the applications, then selects the newer valid application. If both applications are invalid, the boot will be aborted. Then it generates the DeviceID key, Alias key and Alias certificate. Here the Alias key generation depends on the hash of either slot A or B, depending on which slot gets booted. A custom certificate extension is used in the Alias certificate generation which contains a reference to the selected manifest and important data from the manifest. This includes:

- A fingerprint of the public key used for manifest verification, which identifies the signer of the manifest
- Sequence number of the manifest, which is a monotonically increasing number for anti-rollback
- Digest of the firmware image
- Slot of the firmware image
- Reference URI, which enables the certificate recipient to retrieve the manifest from the update server

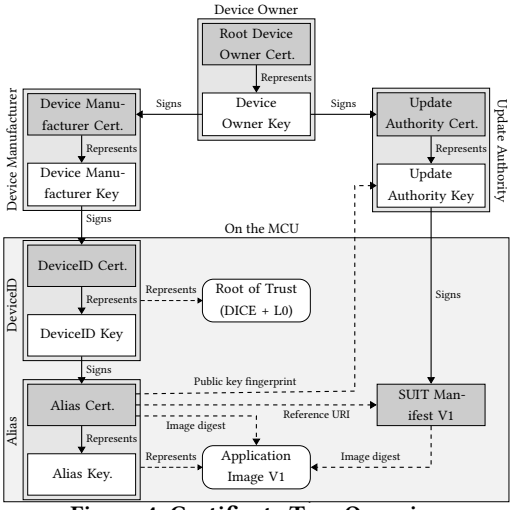

Figure 4: Certificate Tree Overview

This enables the certificate recipient to check the manifest to decide whether they want to trust the application. The public key(s) used for manifest verification may either be fixed for each device to ensure that the device will only execute applications from certain sources or may be configurable for better flexibility. In the latter case the certificate can be used to verify who the signee of the currently running application is. The resulting certificate tree can be seen in Fig. 4.

The verification of two manifests and the generation of two asymmetric keys and a certificate takes a long time on low-level MCUs, but optimizations allow us to skip these steps for most of the boots, which is explained in Section 3.6.

3.4 HTA Authorization & Remote Attestation

The attestation capabilities of DICE can be used both for HTA authorization and for classical DICE Remote Attestation. Here the manifest data included in the certificate can for example be used to implement downgrade protection. Additionally, it acts as endorsements as the certificate contains information on the signee of the currently executed application, so the verifier can check whether a trusted source signed the update. Additional checks may be executed by using the reference URI to retrieve the manifest.

To defend against runtime attacks, HTA authorization should be performed before activating network interfaces and attestation keys should be deleted from the RAM afterward. This ensures that an attacker does not get access to attestation keys, which would permanently compromise the system. With this countermeasure, they may access sealed data of the MCU, but they cannot access keys and other data secured by the HTA. To stop an attacker from never rebooting the system, the HTA should require fresh authorizations and thus reboots in regular intervals if possible.

For normal Remote Attestation, attestation cannot be performed before activating the network interface. One alternative would be to do attestation as the first thing after activating the interface, and replay protection then should be implemented using a counter instead of a nonce. More details are given in [17], which also explains alternative countermeasures.

Normal Remote Attestation has the advantage that a verifier with an independent network connection can detect when an available update has not been installed for a while. On the other hand, attestation based on the HTA is more effective in enforcing downgrade protection, can enforce frequent reboots to ensure freshness and is easier to protect against runtime attacks.

3.4.1 Hybrid Attestation. When using DICE to bootstrap an HTA, asymmetric attestation may be infeasible to do on every boot because it slows down the boot considerably, depending on the processing power of the MCU and HTA. However, symmetric attestation does not work with updates. For these cases, we created a hybrid attestation method. Hybrid attestation basically uses symmetric attestation when possible, but falls back to a modified asymmetric attestation when necessary, which is shown in Fig. 5. In the asymmetric attestation process, the new symmetric attestation key gets shared with the HTA, so that symmetric attestation can be used for the next boot. As the asymmetric attestation ensures that the MCU currently is in a trustworthy state, the MCU can be trusted to provide the new symmetric attestation key correctly. As long as the symmetric key does not change, we know that the firmware did not get changed.

For asymmetric attestation, verification is based on verifying both the trustworthiness and the possession of the attesters Alias key. Usually, this is done by verifying the certificate chain of the Alias key and challenging the attester to sign a challenge using the Alias key. For our hybrid attestation, we swap out the latter by an ECDH key exchange as we require a session key. The key exchange can only be performed if the attester has access to the Alias key, so a successful key exchange is a proof of possession of the Alias key for the attester. The session key gets used for an authenticated encryption scheme, in our case AES-GCM, where the attester encrypts and sends the current symmetric attestation key to the verifier.

3.5 Additional Options

3.5.1 Downgrade Protection. Downgrade protection for the MCU firmware can be implemented on the HTA. In the event of a downgrade, the HTA would refuse authorization of the MCU until a successful attestation with a newer version is performed. This comes with the advantage that the MCU may roll back the firmware version in case of a bug in the new firmware version, but may not access the HTA until it had upgraded to a newer version again.

3.5.2 Sealed Data. Additional to the isolated memory of the HTA, the MCU may seal data in its flash memory for HTA-independent access. Only valid firmware images may access this sealed memory, which is ensured by L0. Here L0 generates the symmetric sealing key based on the CDI_{L0} and only provides this key to up-to-date and valid firmware images. A disadvantage of this approach is that a runtime attacker may get permanent access to sealed data, contrary to data secured by the HTA, where access will be revoked after the next reboot, but this disadvantage is inherent to all sealing approaches that uses DICE.

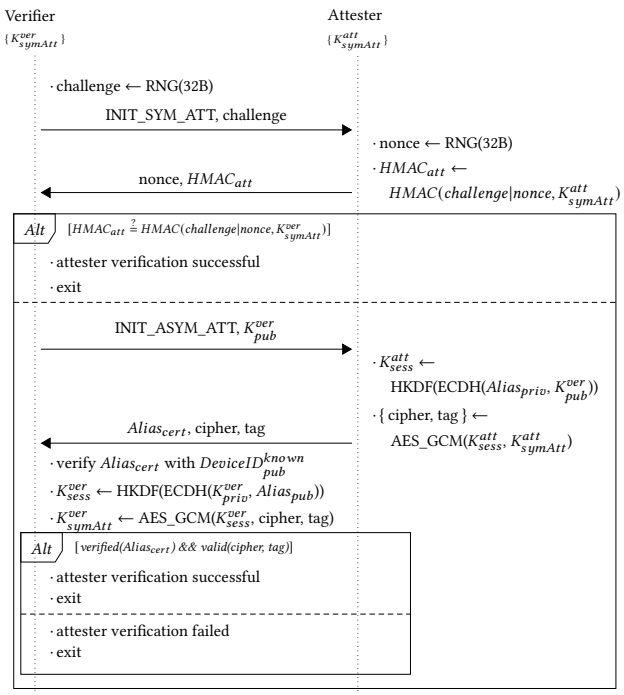

Figure 5: Hybrid Attestation Sequence Diagram

3.6 Optimizations

Both the asymmetric key generation and the manifest parsing and verification take a long time on resource-constrained devices due to their usage of asymmetric cryptography. This might be a problem in environments where a quick boot is required. Optimizations can be used here to lessen the impact of these steps.

3.6.1 Asymmetric Key Caching. The first optimization is caching keys to avoid asymmetric key generation if it is not needed. This can be implemented based on sealing. Here each asymmetric key and certificate gets saved in non-volatile memory using an authenticated encryption scheme, with a symmetric key that is derived from the same inputs as those used to generate the asymmetric key. On following boots the system will be able to decrypt the key and certificate if the inputs used for the key generation did not change. This accelerates the boot if there was no update of firmware or manipulation of sealed data since the last boot. In our case, we used AES-GCM based on AES HW acceleration, but the impact of the HW acceleration should be low as the amount of data to encrypt and decrypt is low (less than 250 bytes for a DER-encoded key). For comparison, during each boot at least 150 kB of firmware get hashed in our implementation, which causes a much larger impact on the runtime.

3.6.2 Application Slot Caching. The second optimization involves the manifest verification. Here the parsing and verification, which both involve asymmetric cryptography, can be avoided if neither the manifest nor the firmware image changed since the last boot. This gets checked using hashes of manifest and image, which are cached in non-volatile memory, integrity-protected using an HMAC. Additional metadata (like the sequence number) also gets cached. The algorithm shown in Algorithm 1 is optimized to avoid asymmetric cryptography and hashing operations where possible. If no update was installed since the last boot, then only two manifests and one firmware image need to be hashed to verify that they did not change. After an update, two manifests need to be hashed and one manifest needs to be parsed and verified (which includes hashing the firmware image).

Note that the algorithm uses active slot and inactive slot as terminology. The active slot refers to the slot that was booted at the last boot and thus is also assumed to be booted this time if neither a manifest nor the boot slot image changed. The inactive slot is the other slot. This terminology represents the assumption at the start of the algorithm, so the inactive slot may be booted, becoming the active slot, if a manifest or image changed due to an update.

Algorithm 1
Optimized A/B slot processing with SUIT Manifests

```
mfstData ← readCachedMfstData();
if valid(mfstData) then
    // act = active slot, inact = inactive slot.
    // Implicit conversion from act / inact to slotA / slotB
       assumed.
    inact.status ← checkMfstHash(inact.mfst,
      mfstData.inact.mfstHash);
    if inact.status = CHANGED then
        inact.status, mfstData.inact ←
          processMfst(inact.mfst);
    act.status ← checkMfstHash(act.mfst,
      mfstData.act.mfstHash);
    if act.status = CHANGED then
        act.status, mfstData.act ← processMfst(act.mfst);
    else if inact.status ≠ NEWER_VALID then
        act.status = checkImageHash(act.image,
          mfstData.act.imageHash);
    // Only needed if downgrade protection is not used.
    if act.status ≠ NEWER_VALID and inact.status =
      UNCHANGED then
        inact.status = checkImageHash(inact.image,
          mfstData.inact.imageHash);
else
    mfstData.slotA ← processMfst(slotA.mfst);
    mfstData.slotB ← processMfst(slotB.mfst);
if changed(mfstData) then
    writeCachedMfstData(mfstData);
```

4 IMPLEMENTATION

Our prototype implementation implements all concepts introduced in this paper, but only for the MCU. The HTA is implemented conceptually using python scripts on a PC. This was chosen because most concepts of this paper are concerning the MCU and not the HTA. The only exception here is the hybrid attestation, which impacts both the MCU and HTA. Also, development boards integrating both a MCU and a freely programmable HTA are often

Table 1: Code Sizes of components

Component	Code Size (B)	
	PKA	Non-PKA
DICE	15972	15972
L0	61492	68940
App	89256	95328

Table 2: Total Boot Delay for different kinds of boots

Boot Scenario	Total Boot Delay (ms)	
	PKA	Non-PKA
First Boot	968.3	1836.8
App Update	596.2	1064.6
No change	198	214.2

not easily available to research institutions (except for TrustZone, but we are targeting a different kind of HTA here). The update server is also implemented conceptually using python scripts on a PC.

For the MCU, we used a Nucleo-WB55RG, a development board from STMicroelectronics featuring their STM32WB55RG MCU, based on an ARM Cortex-M4 running at 64 MHz. The MCU also features an ARM Cortex-M0+ as coprocessor for real-time radio operation using Bluetooth Low Energy and IEEE 802.15.4, but it was not used for our prototype as we used UART for communication to keep it simple. Upon booting, the main application core of the MCU executes DICE and L0. After booting the application, it waits for requests to receive a manifest or to perform attestation / authentication via UART. The MCU provides 256 KB of SRAM, but we limited DICE to 8 KiBs and L0 to 24 KiBs to show that the concept also can be implemented on smaller MCUs. The MCU features a True Random Number Generator (TRNG), two AES accelerators and a Public Key Accelerator (PKA). We used one of the AES accelerators to implement asymmetric key generation caching based on Sealing and our hybrid attestation mode. We have evaluated the system with and without public key acceleration.

The firmware for the MCU was implemented in C. We used MbedTLS [3] (version 3.2.1) as cryptographic library. For manifest processing, L0 and the application use a library called libcsuit [28] (commit 77787c9 from Nov' 22, with small custom modifications). It provides the functionality to parse manifests. A manifest processor is also provided, but it was not usable at the time of implementation for our use case, so we implemented our limited manifest processor based on the parsing code of libcsuit. Our processor implementation is limited in its features to the features we require, for example dependencies to other manifests are not supported. Manifests were created using suit-tool [4] (commit 7886ab28), a Python tool for creating and parsing manifests.

5 EVALUATION

In this section, we first evaluate Performance, Security and Feasibility. A summary concludes this.

5.1 Performance Evaluation

For the evaluation, we measured the runtime and code sizes of different components. We measured both with and without Public Key Accelerator (PKA). AES acceleration was used in all cases, but AES generally makes up only a minimal portion of the total runtime, so devices without AES acceleration should perform similarly. This is because AES input sizes are small when compared to hashing operations (AES encrypts and decrypts less than 1 000 bytes during a boot, while at least 150 000 bytes get hashed during a boot).

We measured runtime both internally and externally. The latter was used for the total boot delay, which describes the delay from releasing the reset signal of the MCU to having a fully booted application. This is measured using a second MCU, which releases the reset signal of the primary MCU and then waits for a GPIO pin activation. The GPIO pin gets activated by the application of the primary MCU after it has fully booted. The delay gets measured using a hardware timer with a resolution of 1 μs. Internal measurements were done using a hardware timer of the primary MCU, again with the same resolution, to be able to measure specific sections of code.

5.1.1 Memory Requirements. The code size for all components is given in Table 1. Both L0 and the application use MbedTLS and libcsuit to implement their functionality.

We did not directly evaluate RAM sizes, but limited DICE to using 8 KiBs and L0 to using 24 KiBs using linker scripts to show that RAM requirements should not be an obstacle for most MCUs.

5.1.2 Total Boot Delay. The total boot delay is given in Table 2. It was measured for three different scenarios, the first boot, a boot after an update to the application and a boot where no component changed since the last boot. The differences in runtime are entirely caused by L0, which is influenced by whether sealed data is available or not. For the first boot, the DeviceID key, the Alias key and an Alias certificate get generated and both the manifests for slot A and B are getting parsed and verified. In the app update scenario, the Alias key and Alias certificate get generated and one manifest gets verified. For the no change scenario, both the key generation and the manifest verification can be avoided due to the caching method introduced in Section 3.6. In this case, hashing operations on both manifests and the image to boot verify that neither of them changed.

It can be seen that the Public Key Accelerator (PKA) primarily impacts the boot delay when some part of the system changed. Only symmetric cryptography is used in the no change scenario, so PKA does not have a direct impact in this case. Here the boot delay is slightly smaller because code sizes are smaller when using the PKA.

5.1.3 DICE & L0 Boot Time. The detailed boot times of DICE and L0 were measured internally. Table 3 evaluates each step of the DICE and L0 execution. For DICE, boot times are dominated by hashing the L0 binary with more than 97% of the total runtime. The rest of the runtime is consumed by generating the HMAC and reading out the UDS. The PKA version is faster because the L0 binary is smaller when using PKA.

The first step of L0 is checking whether cached manifest data is valid. In the first boot scenario the data is invalid, so it gets erased, and both manifests get parsed normally. Then the updated manifest

Table 3: Boot times for all three scenarios

	Runtime (ms)					
	PKA			Non-PKA		
Step	First Boot	App Upd.	No Cha.	First Boot	App Upd.	No Cha.
DICE: Boot & generate CDI	73.8	73.8	73.8	82.6	82.6	82.6
L0-1: Read cached manifest data	22.4	1.5	1.5	22.4	1.5	1.5
L0-2: Check non-boot slot digest		0.5	0.5		0.5	0.5
L0-3: Process non-boot slot manifest	274.4	274.4		614.6	614.6	
L0-4: Check boot slot digests		0.5	108.5		0.5	115.6
L0-5: Process boot slot manifest	269.3			614.4		
L0-6: Write cached manifest data	4.8	26.6		4.8	26.6	
L0-7: Generate DeviceID key	78.9			129.3		
L0-8: Sealing overhead	25.3			25.3		
L0-9: Unseal DeviceID key		0.8	0.8		0.8	0.8
L0-10: Generate Alias key & cert	164.7	162.4		273.8	276.4	
L0-11: Sealing overhead	29.2	29.4		29.2	29.4	
L0-12: Unseal Alias key & cert			1.0			1.0
L0-13: Generate sym. attestation key	0.8	0.8	0.8	0.8	0.8	0.8

Table 4: Attestation runtime measurements

	Runtime (ms)	
Step	PKA	Non-PKA
RNG: Generate 32B	0.1	0.1
Calculate HMAC	0.5	0.5
Sym. att. comm.	13.4	13.8
Import Alias key	0.3	0.1
ECDH key exchange	83	348.5
Encrypt sym. att. key	0.4	0.4
Asym. att. comm.	41.9	43.3

Table 5: Runtime measurements for installing updates

	Runtime (ms)	
Step	PKA	Non-PKA
Receive manifest	34.5	34.2
Decode & verify manifest	165.2	499.6
Install: Download image	10116.8	10704.1
Install: Hash image	106.2	113.1

data gets written to the caching section. Afterward, the DeviceID key, Alias key and Alias certificate get generated. These get sealed and written to flash memory, with most of the sealing overhead being made up of needing to erase the corresponding flash sections before writing the data. Finally, the symmetric attestation key gets generated.

In the app update scenario, the cached manifest data is valid. Here the inactive slot manifest first gets hashed, which detects a changed manifest, so the manifest gets parsed. The new manifest is detected as valid and newer than the previous manifest, so it gets accepted. Then the active slot manifest gets hashed which matches the cached data, so we know that the manifest from this slot is older than the new manifest. The updated manifest data gets written to the caching section. The DeviceID key gets unsealed, while the Alias key and certificate get generated and written to the sealing section. Lastly, the symmetric attestation key gets generated again and the previously inactive slot gets booted.

When there was no update of the system, most of these steps can be skipped. The cached manifest data gets detected as valid again. The inactive slot manifest and the active slot manifest & image get checked using their hashes. As they did not change, the active slot can be booted without parsing any manifests. The DeviceID key, Alias key and Alias certificate also can be unsealed, only the symmetric attestation key needs to be generated.

The results prove that the optimizations from Section 3.6 considerably speed up the boot while causing small overheads in cases where the cached data needs to be re-generated. The overheads are small enough that even the app update benefits from the optimizations although more than half of the optimizations cannot be executed as the cached data is outdated. This can be seen when comparing the runtimes of the app update case against L0 without any optimizations: L0 without optimizations takes 1632.9 ms (non-PKA, theoretical calculation by summing up all runtimes from the first boot scenario while removing steps only necessary due to the optimizations), while the optimized app update scenario takes 951.1 ms and the no change scenario takes 129.2 ms.

5.1.4 Application Runtime. The application runtime is measured for both the hybrid attestation and the update case. All measurements were done exclusively on the MCU, runtime of the update server and verifier is out of scope. Due to the proof-of-concept implementation based on UART communication, all parts involving communication should only be used as rough estimates. For the attestation, these parts are separated from runtime of other parts. For the update case, the download & write to flash step contains both the communication and the writing of received data to flash as these are executed in parallel. The 'receive update' step for the update case contains only communication delay.

Table 4 contains both the runtime of the symmetric cryptography steps and the following asymmetric parts. The asymmetric parts only need to be executed if an update or some other change happened since the last attestation. We assumed that the DeviceID is already trusted for the asymmetric parts. Otherwise, the DeviceID certificate from the manufacturer would also need to be retrieved from the device.

Runtimes for the update installation are shown in Table 5. The PKA primarily accelerates the manifest verification step as signature verification is faster, but smaller image sizes due to using the PKA also reduce runtime of downloading, writing to flash and hashing operations.

5.2 Security Evaluation

Three different parts are critical for the security of the system, the update architecture, DICE-based boot and HTA authentication & attestation. We evaluate their security in this section.

5.2.1 Update Architecture. SUIT Manifests are in the process of becoming a public standard; therefore they benefit from public reviews and widespread collaboration. Currently, SUIT Manifests are still used relatively rarely, but we expect them to become a common standard. Standardization work is performed by ARM, Fraunhofer SIT, Inria and Nordic Semiconductors [24], while an experimental implementation is available for RIOT OS [1], an IoT operating system. Furthermore, the flexibility of the SUIT update architecture allows it to be directly mapped to DICE devices, allowing DICE and non-DICE devices to use the same SUIT Manifest processor. This reduces likeliness of bugs as more code can be re-used.

5.2.2 DICE-based Boot. The boot is secured by DICE and L0 integrated with SUIT Manifests. The security level is similar to normal DICE systems, with some additional options to stop the boot of manipulated firmware. The basic concept will stop the boot if the application firmware has been manipulated. This can be circumvented by also manipulating L0 code, but then the entire MCU will be re-keyed, losing access to keys, sealed data and the HTA.

By integrating L0 and DICE into a single component, a true secure boot could be achieved. Then L0 no longer could be updated, but this is already only possible in a secure environment.

5.2.3 HTA Authentication & Attestation. The hybrid attestation can be considered to have the same security strength as normal DICE as long as the security strength of involved cryptography meets the same standards as normal attestation. After updates, the modified asymmetric attestation is performed just as normal asymmetric attestation, with the only difference that the proof of possession of the Alias key is performed using ECDH key exchange instead of a signature. The session key then gets used to encrypt the symmetric attestation key. Here an authenticated encryption scheme with the same security strength as the keys is required. For the following boots, symmetric attestation will be used that has the same security strength as the asymmetric attestation.

In the HTA authentication scenario, the authentication is additionally secured by the separation of the system. Some attack vectors from Remote Attestation via networks do not apply here as the attestation is typically performed on a bus that no other component has access to. So, the biggest remaining risk is that compromises of the MCU are not detected by the attestation. DICE is limited in this regard as runtime attacks pose a risk to permanently compromise the system if they manage to access attestation keys. However, a simple countermeasure here would be to only enable network interfaces after authentication has been performed and the attestation keys have been erased from RAM. Repeating authentication at a later point makes no sense as DICE attestation only attests the status of the system during boot. For general purpose attestation, other countermeasures might be more suitable depending on the scenario, see the work by Hristozov et al. [18].

5.3 Feasibility Evaluation

SUIT Manifests have the advantage that the steps of manifest creation, update delivery, update processing and installation can be shared across DICE and non-DICE devices, which unifies the architecture and makes adoption of DICE into an existing update infrastructure more likely. Libraries for manifest creation, processing, and update installation already exist, which makes implementing updates with manifests easier. Reusing code should also make implementations more secure as there is less potential for bugs than in custom implementations. The flexibility of SUIT Manifests enable them to cover many update architectures. In our case, we use an A/B image structure.

Our concept for HTA authentication requires asymmetric cryptography and an authenticated encryption scheme on the HTA. If the latter is not available, normal asymmetric attestation can be used for authentication, but this will cause a bigger overhead in terms of boot time. Asymmetric cryptography is required in every case, but this is inherent to updates using manifests and updatable DICE-based systems.

5.4 Summary

Overall, our concept improves the security of HTA-enabled devices without requiring any additional hardware. Most parts of the concept are already standardized and well-known, making bugs and flaws unlikely and adoption easier if, e.g.; DICE or SUIT Manifests are already used for other purposes. A risk remains that the complex interplay between systems causes flaws and bugs, but the risk here should be reasonable as most parts should already be part of a secure embedded system. This includes an update architecture based on manifests and a DICE- or Secure-Boot-based boot architecture that integrates updates.

6 CONCLUSION

This paper has presented a new concept for implementing authentication for HTAs. This can be used to secure HTAs in any field where security is critical. Examples could be safety-critical fields and critical infrastructure.

The evaluation show promising results as there is almost no overhead in terms of runtime if there has not been an update since the last boot. After updates, there is some overhead, but some delay after update installation should be acceptable as downtime is expected anyway. There is some overhead in terms of code size, but other systems with A/B slots will have similar requirements. Overall significant security benefits can be achieved. DICE architecture implementations without an HTA available also can benefit from the update architecture, DICE-based boot system with manifests and Remote Attestation mechanisms provided in this paper.

Regarding general DICE implementations, future work could explore how to integrate a device management protocol like the Lightweight Machine-to-Machine (LwM2M) protocol [29] or the DICE endorsement architecture [15]. Regarding HTA-enabled devices, an in-depth comparison with the DICE Protection Environment (DPE) [16] could be done.

ACKNOWLEDGMENTS

This research work was supported by the National Research Center for Applied Cybersecurity ATHENE. In addition, the work was funded by the German Federal Ministries BMBF and BMWK as part of the projects HardShiP (ID 16KIS1860), RESISTANT (ID 20D2116B), ProSeCA (ID 19A23009F) and ELISA (ID 19S23002E).

REFERENCES

[1] [n. d.]. RIOT OS. https://www.riot-os.org/.

[2] 1997. HMAC: Keyed-Hashing for Message Authentication. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc2104

[3] Arm. 2023. MbedTLS. https://www.trustedfirmware.org/projects/mbed-tls. Accessed: 2023-03-16.

[4] Arm. 2023. suit-tool. https://gitlab.arm.com/research/ietf-suit/suit-tool. Accessed: 2023-04-12.

[5] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. 1996. Keying Hash Functions for Message Authentication. In *Advances in Cryptology — CRYPTO '96*, Neal Koblitz (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–15.

[6] Carsten Bormann and Paul E. Hoffman. 2020. Concise Binary Object Representation (CBOR). RFC 8949. https://doi.org/10.17487/RFC8949

[7] Paul England, Andrey Marochko, Dennis Mattoon, Rob Spiger, Stefan Thom, and David Wooten. 2016. *RIoT - A Foundation for Trust in the Internet of Things*. Technical Report MSR-TR-2016-18. https://www.microsoft.com/en-us/research/publication/riot-a-foundation-for-trust-in-the-internet-of-things/

[8] Robert Escherich, Ingo Ledendecker, Carsten Schmal, Burkhard Kuhls, Christian Grothe, and Frank Scharberth. 2009. *SHE – Secure Hardware Extension Functional Specification Version 1.1, Rev: 439*. Technical Report.

[9] Trusted Computing Group. 2018. *Hardware Requirements for a Device Identifier Composition Engine*. Technical Report. Trusted Computing Group.

[10] Trusted Computing Group. 2018. *Implicit Identity Based Device Attestation*. Technical Report. Trusted Computing Group.

[11] Trusted Computing Group. 2019. *Trusted Platform Module Library Specification, Family "2.0", Level 00, Revision 01.59*. Technical Report. Trusted Computing Group.

[12] Trusted Computing Group. 2020. *DICE Layering Architecture*. Technical Report. Trusted Computing Group.

[13] Trusted Computing Group. 2020. *Symmetric Identity Based Device Attestation*. Technical Report.

[14] Trusted Computing Group. 2021. *DICE Attestation Architecture*. Technical Report. Trusted Computing Group.

[15] Trusted Computing Group. 2022. *DICE Endorsement Architecture for Devices*. Technical Report. Trusted Computing Group.

[16] Trusted Computing Group. 2023. *DICE Protection Environment*. Technical Report. Public Review.

[17] Stefan Hristozov, Johann Heyszl, Steffen Wagner, and Georg Sigl. 2018. Practical Runtime Attestation for Tiny IoT Devices. In *NDSS Workshop on Decentralized IoT Security and Standards (DISS) 2018, San Diego, CA, USA*. San Diego, CA, USA. https://doi.org/10.14722/diss.2018.23011

[18] Stefan Hristozov, Moritz Wettermann, and Manuel Huber. 2022. A TOCTOU Attack on DICE Attestation. In *Proceedings of the Twelveth ACM Conference on Data and Application Security and Privacy*. ACM. https://doi.org/10.1145/3508398.3511507

[19] Manuel Huber, Stefan Hristozov, Simon Ott, Vasil Sarafov, and Marcus Peinado. 2020. The Lazarus Effect: Healing Compromised Devices in the Internet of Small Things. *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security* (Oct 2020). https://doi.org/10.1145/3320269.3384723

[20] Rhys Kirk, Hoang Nga Nguyen, Jeremy Bryans, Siraj Ahmed Shaikh, and Charles Wartnaby. 2023. A formal framework for security testing of automotive over-the-air update systems. *Journal of Logical and Algebraic Methods in Programming* 130 (2023), 100812. https://doi.org/10.1016/j.jlamp.2022.100812

[21] Trishank Kuppusamy, Lois DeLong, and Justin Cappos. 2018. Uptane: Security and Customizability of Software Updates for Vehicles. *IEEE Vehicular Technology Magazine* PP (02 2018), 1–1. https://doi.org/10.1109/MVT.2017.2778751

[22] Paolo Matarazzo and Vinay Pamnani. 2023. BitLocker Countermeasures. https://learn.microsoft.com/en-us/windows/security/operating-system-security/data-protection/bitlocker/bitlocker-countermeasures.

[23] Brendan Moran, Hannes Tschofenig, and Henk Birkholz. 2022. A Manifest Information Model for Firmware Updates in Internet of Things (IoT) Devices. RFC 9124. https://doi.org/10.17487/RFC9124

[24] Brendan Moran, Hannes Tschofenig, Henk Birkholz, Koen Zandberg, and Øyvind Rønningstad. 2023. *A Concise Binary Object Representation (CBOR)-based Serialization Format for the Software Updates for Internet of Things (SUIT) Manifest*. Internet-Draft draft-ietf-suit-manifest-22. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-suit-manifest/22/ Work in Progress.

[25] Brendan Moran, Hannes Tschofenig, David Brown, and Milosch Meriac. 2021. A Firmware Update Architecture for Internet of Things. RFC 9019. https://doi.org/10.17487/RFC9019

[26] Christian Plappert, Dominik Lorych, Michael Eckel, Lukas Jäger, Andreas Fuchs, and Ronald Heddergott. 2023. Evaluating the applicability of hardware trust anchors for automotive applications. *Computers & Security* 135, 103514. https://doi.org/10.1016/j.cose.2023.103514

[27] Jim Schaad. 2017. CBOR Object Signing and Encryption (COSE). RFC 8152. https://doi.org/10.17487/RFC8152

[28] Secom. [n. d.]. libcsuit. https://github.com/kentakayama/libcsuit. Accessed: 2023-04-12.

[29] OMA SpecWorks. [n. d.]. LightweightM2M. https://technical.openmobilealliance.org/index.html. Accessed: 2023-04-12.

[30] Marko Wolf and Timo Gendrullis. 2012. Design, Implementation, and Evaluation of a Vehicular Hardware Security Module. In *Information Security and Cryptology - ICISC 2011*, Howon Kim (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 302–318.

[31] Meng Xu, Manuel Huber, Zhichuang Sun, Paul England, Marcus Peinado, Sangho Lee, Andrey Marochko, Dennis Mattoon, Rob Spiger, and Stefan Thom. 2019. Dominance as a New Trusted Computing Primitive for the Internet of Things. In *2019 IEEE Symposium on Security and Privacy (SP)*. 1415–1430. https://doi.org/10.1109/SP.2019.00084

[32] Koen Zandberg, Kaspar Schleiser, Francisco Acosta, Hannes Tschofenig, and Emmanuel Baccelli. 2019. Secure Firmware Updates for Constrained IoT Devices Using Open Standards: A Reality Check. *IEEE Access* 7 (2019), 71907–71920. https://doi.org/10.1109/ACCESS.2019.2919760

Received 13 March 2024; accepted 3 May 2024